

Seçilen Problem

(4. Grup + 10 bonus puan) Concav yada convex bir poligon olacak şekilde belirli x, ve y koordinat ikilileri verilmiştir. Bir noktanın bu poligonun içinde olup olmadığını parallel proqramlama ile bulunuz.

GitHub Proje Linki:

<https://github.com/AFurkanOcel/Parallel Point In Polygon>

Java ile Paralel Point-In-Polygon Algoritması ve Speedup Analizi

Bu raporda, Java programlama dili ile geliştirdiğim paralel Point-In-Polygon uygulamasının çalışma prensibini, kullandığım fonksiyonları, paralelleştirme stratejisini ve benchmark sonuçlarını açıklıyorum. Projedeki temel amaç, çok sayıda noktanın verilen bir polygon içinde olup olmadığını belirlemek ve bu işlemi sequential çözüm ile paralel çözüm arasında karşılaştırmaktır.

Point-In-Polygon problemi özellikle coğrafi bilgi sistemleri, bilgisayar grafikleri ve geometrik hesaplama problemlerinde sık kullanılan temel problemlerden biridir. Nokta sayısı arttığında her nokta için polygon kenarlarının kontrol edilmesi gerektiğinden işlem maliyeti büyür. Bu nedenle projede her noktanın bağımsız kontrol edilebilmesinden yararlanarak paralel programlama yaklaşımını kullandım.

1. Problem Tanımı ve Genel Çalışma Prensibi

Programda polygonu, sıralı köşe noktalarından oluşan kapalı bir şekil olarak ele aldım. Her nokta x ve y koordinatlarıyla temsil edilir. Polygon convex veya concave olabilir. Bu projede edge veya vertex üzerinde bulunan noktaları da inside kabul ettim. Bu tercih, sınır üzerinde kalan noktaları polygonun parçası olarak değerlendirdiği için daha tutarlı bir sonuç üretmektedir.

Programın genel akışında önce benchmark için kullanılacak polygon ve rastgele noktaları oluşturuyorum. Daha sonra aynı veri kümesi üzerinde önce sequential çözümü çalıştırıyorum. Ardından aynı noktaları farklı thread sayılarıyla paralel olarak işliyorum. Her çalıştırmada içeride bulunan nokta sayısı ve çalışma süresi ölçülür. Son aşamada speedup değeri hesaplanır, sonuçlar CSV dosyasına yazılır ve grafik oluşturulur.

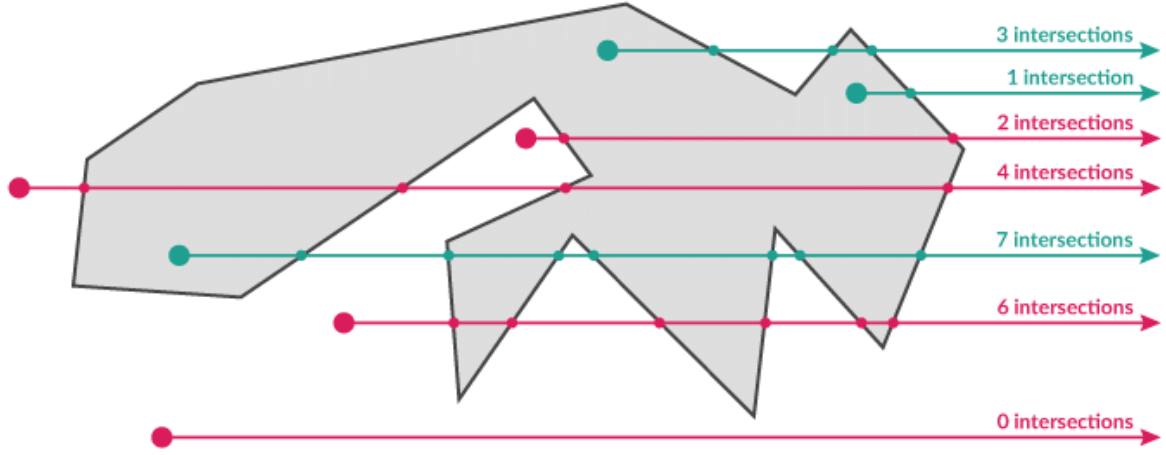
Bu yapıda sequential çözüm referans kabul edilmiştir. Paralel çözümün doğru çalışıp çalışmadığını anlamak için paralel inside count değeri sequential sonuç ile karşılaştırılmıştır. Eğer iki sonuç farklı olursa benchmark başarısız kabul edilir. Böylece sadece hız değil, doğruluk da kontrol edilmektedir.

Program çalıştırıldığında örnek olarak aşağıdaki gibi bir çıktı üretir. Bu çıktı hem inside count değerini hem de thread sayısına göre speedup değerini göstermektedir:

Testing with 1000000 points

Sequential -> Inside: 784968 | Average Time: 589.595 ms

2. Kullanılan Algoritma: Ray Casting



Şekil 1. Ray Casting algoritması

Noktanın polygon içinde olup olmadığını belirlemek için Ray Casting algoritmasını kullandım. Bu algorithmada test edilen noktadan sağa doğru yatay bir ışın gönderildiği varsayılır. Bu ışının polygon kenarlarıyla kaç kez kesiştiği sayılır. Kesişim sayısı tek ise nokta polygon içindedir, çift ise dışındadır.

Bu yaklaşımı seçmemin sebebi, Ray Casting algoritmasının hem convex hem de concave polygonlar için kullanılabilmesi ve her nokta için bağımsız olarak uygulanabilmesidir. Her noktanın sonucu diğer noktalara bağlı olmadığı için algoritma paralelleştirmeye de uygundur.

Algorithmada ayrıca boundary durumları için özel kontrol yapılmıştır. Bir nokta doğrudan polygon kenarı veya köşesi üzerinde bulunuyorsa, ray kesişim sayısına bakmadan inside kabul edilir. Bu durum floating point karşılaştırmalarındaki küçük hassasiyet problemlerini azaltmak için EPSILON toleransı ile desteklenmiştir.

```
if (isPointOnSegment(point, a, b)) {  
    return true;  
}  
if (rayIntersectsSegment(point, a, b)) {  
    inside = !inside;  
}
```

3. Kullanılan Temel Sınıflar ve Fonksiyonlar

Projede kod yapısını basit tuttum, ancak sorumlulukları ayrı sınıflara böldüm. Point sınıfı bir noktanın koordinatlarını immutable biçimde tutar. Polygon sınıfı vertex listesini saklar ve listenin dışarıdan değiştirilmesini engeller. Bu tercih, paralel çalışmada veri güvenliğini artırır.

PointInPolygon.isInside fonksiyonu Ray Casting algoritmasının ana uygulamasıdır. BenchmarkRunner.runSequential fonksiyonu tüm noktaları tek thread üzerinde kontrol eder. BenchmarkRunner.runParallel fonksiyonu ise nokta listesini thread sayısına göre parçalara ayırır ve her parçayı WorkerTask ile işler.

Sequential çözümde temel işlem her nokta için polygon içinde olup olmadığını kontrol edip inside count değerini artırmaktır:

```
if (PointInPolygon.isInside(point, polygon)) {  
    insideCount++;  
}
```

Paralel çözümde ise Java ExecutorService kullanılmıştır. Thread yönetimini doğrudan manuel yapmak yerine ExecutorService kullanmak, kodun daha okunabilir ve kontrollü olmasını sağlamıştır:

```
ExecutorService executor = Executors.newFixedThreadPool(threadCount);  
futures.add(executor.submit(new WorkerTask(points.subList(start, end), polygon)));
```

4. Paralel Çözümün Açıklanması

Paralel çözümde noktaları thread'lere bölerek her thread'in farklı bir nokta grubunu işlemesini sağladım. Bu yaklaşımı seçmemin sebebi, bir noktanın polygon içinde olup olmadığının diğer noktalardan bağımsız hesaplanabilmesidir. Bu nedenle karmaşık senkronizasyon mekanizmalarına ihtiyaç duyulmamıştır.

Her WorkerTask kendisine verilen point chunk üzerinde çalışır ve yalnızca kendi local inside count değerini döndürür. Ana thread, Future nesneleri üzerinden bu kısmi sonuçları toplar. Paylaşılan mutable state kullanılmadığı için race condition riski azaltılmıştır.

Thread sayısı 1, 2, 4 ve 8 olarak denenmiştir. Böylece thread sayısı arttıkça çalışma süresinin ve speedup değerinin nasıl değiştiği gözlemlenmiştir. Paralel çözümde tek thread kullanıldığında bile ExecutorService ve task yönetimi maliyeti bulunduğu için bu durum doğrudan sequential çözümle aynı kabul edilmemelidir.

5. Benchmark Metodolojisi

Benchmark kısmında ölçümlerin daha güvenilir olması için sabit random seed kullandım. Böylece her çalıştırmada aynı nokta dağılımı üretilebilir. Ayrıca JVM'in ilk çalıştırma etkisini azaltmak için warm-up işlemi yaptım. Her konfigürasyonu üç kez ölçüp ortalama süreyi aldım.

Nokta Sayıları	Thread Sayıları	Vertex	Ölçüm	Warm-up	Seed
500000 - 2000000	1, 2, 4, 8	200	3	1	sabit

Speedup değeri aşağıdaki formül ile hesaplanmıştır:

$$\text{Speedup} = \text{Average Sequential Time} / \text{Average Parallel Time}$$

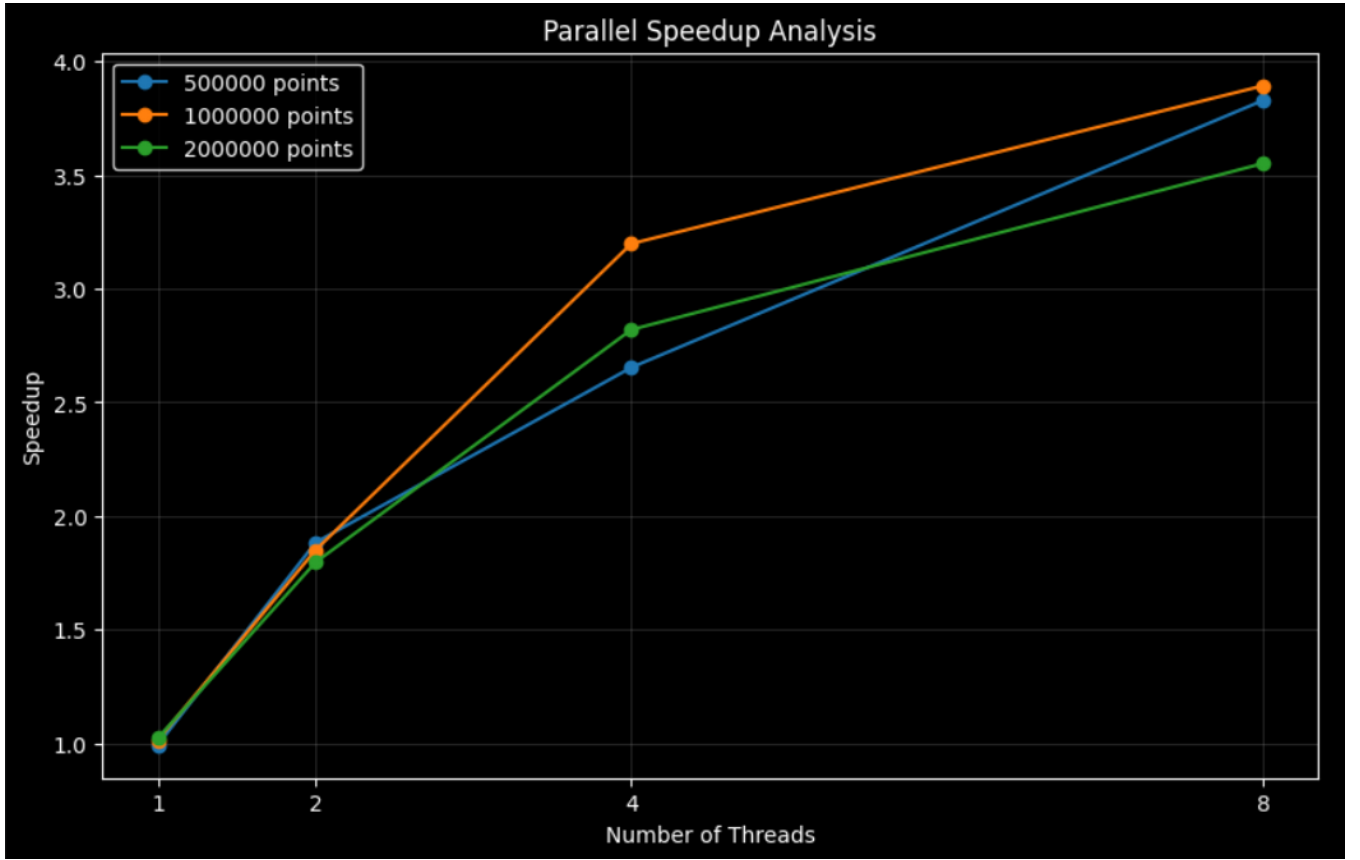
Speedup değerinin 1'den büyük olması, paralel çözümün sequential çözüme göre hızlandığını gösterir. 1'e yakın değerler parallel overhead'in etkili olduğunu, daha yüksek değerler ise paralelleştirmeden daha fazla fayda sağlandığını gösterir.

6. Speedup Sonuçları ve Yorum

Benchmark sonucunda thread sayısı arttıkça genel olarak speedup değerinin arttığı görülmüştür. Ancak artış tamamen lineer değildir. Bunun sebebi thread oluşturma ve yönetme maliyeti, işletim sistemi scheduling etkisi, CPU çekirdek sınırı ve bellek erişim maliyetleri olabilir.

Nokta Sayısı	1 Thread	2 Thread	4 Thread	8 Thread
500000	0.99	1.88	2.65	3.83

1000000	1.01	1.85	3.20	3.89
2000000	1.02	1.80	2.82	3.55



Şekil 2. Hızlanma katsayısı grafiği

Örneğin 500000 nokta için 8 thread kullanıldığında hızlanma katsayısı yaklaşık 3.83 iken, 1000000 nokta için 8 thread kullanıldığında bu değer 3.89 olarak ölçülmüştür. 2000000 nokta için ise 8 thread ile 3.55 hızlanma elde edilmiştir.

En yüksek speedup değeri 1000000 nokta ve 8 thread durumunda yaklaşık 3.89 olarak elde edilmiştir. 1 thread parallel sonuçlarının sequential çözüme çok yakın olması beklenen bir durumdur. Çünkü bu çalışmada parallel tarafta thread pool ve task yönetimi maliyeti de ölçüme dahildir.

7. Doğrulama Senaryoları ve Edge Case Kontrolleri

Programın sadece hızlı değil, doğru sonuç verdiğini göstermek için ValidationTest sınıfını ekledim. Bu sınıfta convex polygon, concave polygon, concave girinti bölgesi, edge üzerindeki nokta, vertex üzerindeki nokta ve sequential-parallel sonuç eşitliği kontrol edilmiştir.

Convex In	Convex Out	Concave In	Concave Gap	Edge	Vertex	Seq-Par
Inside	Outside	Inside	Outside	Inside	Inside	Equal

Bu doğrulamalar için JUnit gibi ek bir framework kullanmadım; çünkü projeyi sade bir akademik Java projesi olarak tutmak istedim. Testler başarısız olduğunda açık hata mesajı üretir, başarılı olduğunda ise tüm validation testlerinin geçtiğini belirtir.

8. Kısıtlar ve Sonuç

Bu çalışmadaki benchmark sonuçları kullanılan donanıma, JVM davranışına ve sistem yüküne göre değişebilir. Ana performans testi circular convex polygon ile yapılmış, concave polygon davranışı ise ayrı validation senaryolarıyla doğrulanmıştır. JMH gibi araçlar kullanılabilir, ancak bu projede sade ve anlaşılır bir ölçüm yapısı tercih ettim.

Sonuç olarak, Point-In-Polygon problemini Java multithreading ile başarılı şekilde paralelleştirdim. Sonuçlar, büyük nokta kümelerinde paralel çalışmanın sequential çözüme göre belirgin hızlanma sağlayabildiğini göstermektedir. Ray Casting algoritması, benchmark sistemi ve validation testleri birlikte değerlendirildiğinde proje ödev gereksinimlerini karşılamaktadır.

Kaynakça

https://en.wikipedia.org/wiki/Point_in_polygon

<https://www.geeksforgeeks.org/dsa/how-to-check-if-a-given-point-lies-inside-a-polygon/>

<https://medium.com/vortehsa/whats-the-point-in-polygon-8e67ded323a2>

<https://www.sanfoundry.com/java-program-check-whether-given-point-lies-given-polygon/>

https://en.wikipedia.org/wiki/Ray_casting

https://rosettacode.org/wiki/Ray-casting_algorithm